

Synchronous Serial Interfaces: SPI and I²C

Introduction to ARM Cortex-M Microprocessors – Chapter 13

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

1 Why This Matters

2 The SPI Bus

3 I²C

4 Design Challenge

1 Why This Matters

2 The SPI Bus

3 I²C

4 Design Challenge

UART connects two devices at modest speed, and it does so without a clock — which is why both ends must agree on the baud rate in advance. Adding a clock wire removes that agreement and, with it, the speed ceiling. This chapter covers the two synchronous buses every microcontroller provides. SPI is fast and simple, and costs a pin per device. I^2C is slower and cleverer, and connects dozens of devices to the same two wires. Both are also where you finally read a real sensor and turn its raw registers into a physical measurement — the complete chain from silicon to engineering units.

After completing this chapter, you should be able to:

1. Identify the four SPI signals and explain how full-duplex transfer works as a circular shift-register exchange.
2. Select the correct clock polarity and phase for a given device, and calculate the SCK frequency from the baud rate prescaler.
3. Configure an SPI master and perform a register read/write transaction.
4. Describe the I^2C start, address, acknowledge, and stop phases, and explain why the bus requires open-drain pins and external pull-up resistors.
5. Configure an I^2C master and read a slave register, selecting the pull-up value from the bus capacitance.
6. Choose between SPI and I^2C for a given set of devices and justify the choice.
7. Interface a real digital sensor and convert its output to engineering units.

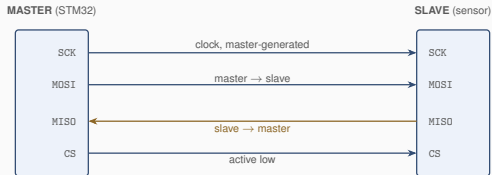
1 Why This Matters

2 The SPI Bus

3 I²C

4 Design Challenge

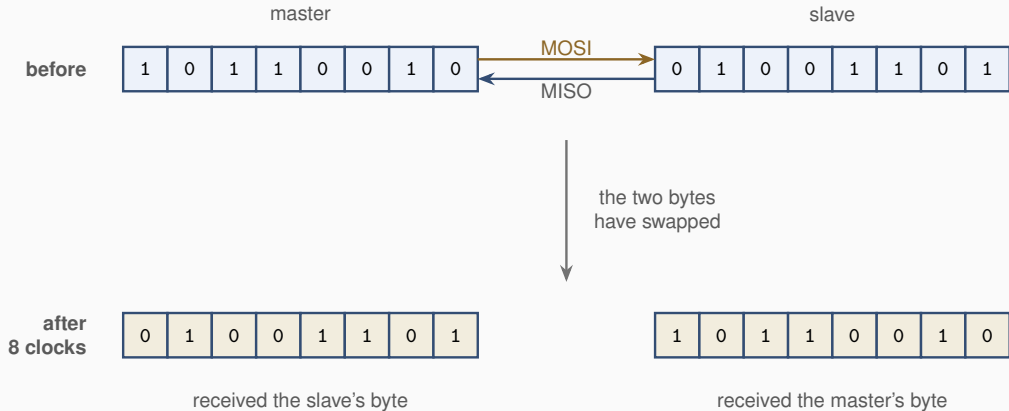
Four Wires, Full Duplex



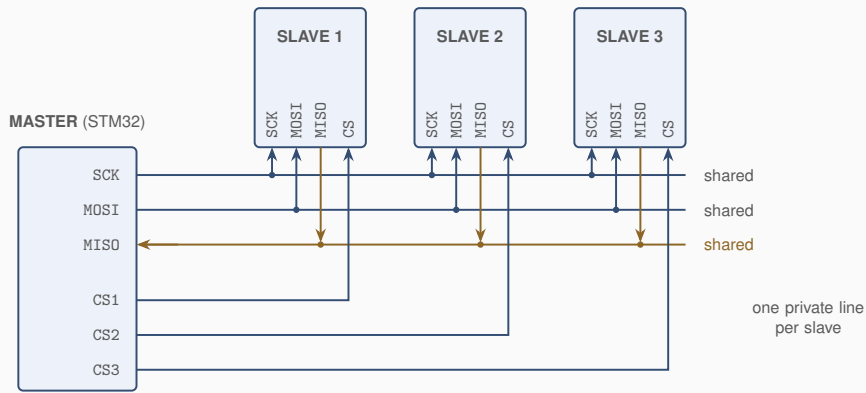
Signal	Direction
SCK	Master → slave
MOSI	Master → slave
MISO	Slave → master
CS	Master → slave, per device

A transfer is an *exchange*, not a send

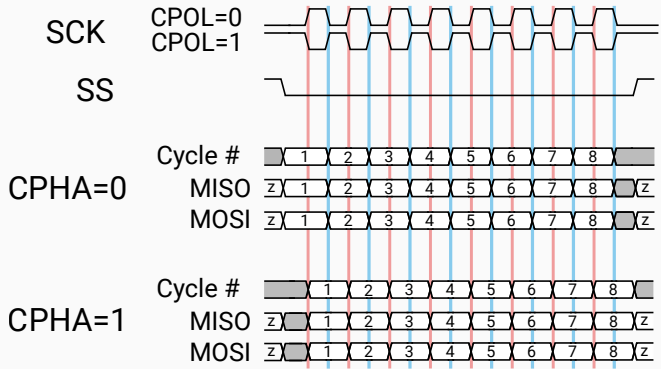
Master and slave are one **circular shift register**. Every clock edge shifts a bit out of each and into the other — so sending a byte *always* receives one at the same time. To read a device you must still **write something** (often a dummy byte) to generate the clock.



SPI is an **exchange**, not a send or a receive. The two shift registers form one ring, so eight clock pulses trade their contents.



SCK, MOSI, and MISO are **shared** by every device; only chip select is **private**. Adding a slave costs exactly one pin.



CPOL sets the idle level of the clock; **CPHA** sets which edge samples the data. Four combinations, called **modes 0 to 3** – and the datasheet tells you which one your device needs.

```
RCC->APB1ENR |= (1U << 14); // SPI2
SPI2->CR1 = (1U << 2)      // MSTR
            | (3U << 3)      // BR: /16
            | (1U << 9)      // SSM
            | (1U << 8);     // SSI
SPI2->CR1 |= (1U << 6);     // SPE
```

$$f_{\text{SCK}} = \frac{f_{\text{PCLK}}}{2^{(\text{BR}+1)}}$$

With $\text{BR} = 3$ and a 16 MHz peripheral clock,
 $f_{\text{SCK}} = 1 \text{ MHz}$.

Why this course uses SPI2, not SPI1

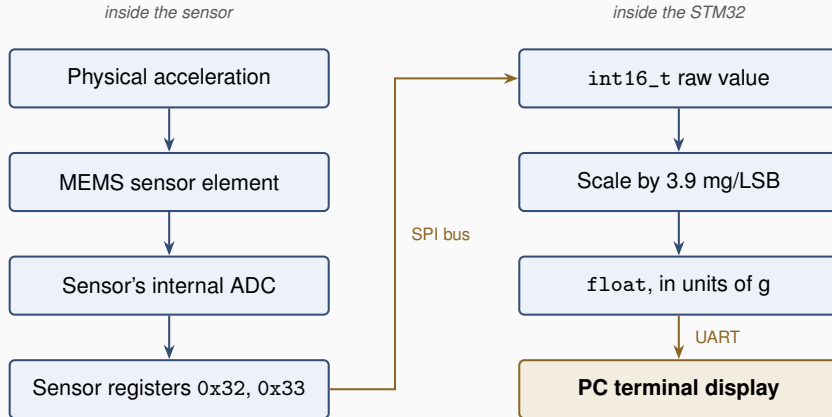
SPI1's default SCK is **PA5** — the user LED. Using SPI1 would silently break every blink program in the book. SPI2 sits on **APB1** (not APB2), and its pins PB13–PB15 live in `AFR[1]`, indexed as $(\text{pin} - 8) \times 4$.

A Register Transaction

```
uint8_t spi_transfer(uint8_t out)
{
    while (!(SPI2->SR & (1U << 1))) ;    // TXE -- ready to send
    *(volatile uint8_t *)&SPI2->DR = out;
    while (!(SPI2->SR & (1U << 0))) ;    // RXNE -- a byte came back
    return *(volatile uint8_t *)&SPI2->DR;
}

uint8_t read_register(uint8_t addr)
{
    cs_low();
    spi_transfer(addr | 0x80);    // bit 7 set = read
    uint8_t v = spi_transfer(0x00); // dummy byte to clock the reply out
    cs_high();
    return v;
}
```

The **dummy byte** is the point: you cannot receive without transmitting, because the master supplies the clock.



From a physical acceleration to a number on a screen. Every stage is a topic from an **earlier chapter**.

1 Why This Matters

2 The SPI Bus

3 I²C

4 Design Challenge

- SCL clock and SDA data — **two** wires total, regardless of device count
- Each device has a **7-bit address**, so no chip-select lines
- Both lines are **open-drain**, with external pull-ups

Why open-drain is mandatory

Many devices share both wires. If two drove push-pull outputs in opposite directions the result would be a **short circuit**.

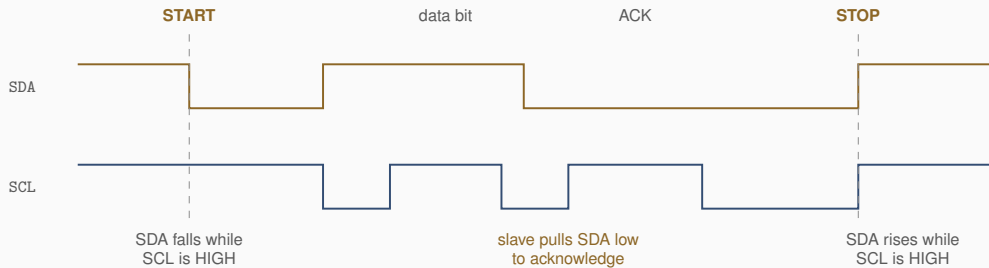
Open-drain means a device can only *pull low* or *release* — so the pull-up defines the idle state and no contention is possible.

That is Chapter 8's open-drain output, now with a reason. The pull-up value follows from **bus capacitance**: too large and the rising edge is too slow; too small and it wastes current.

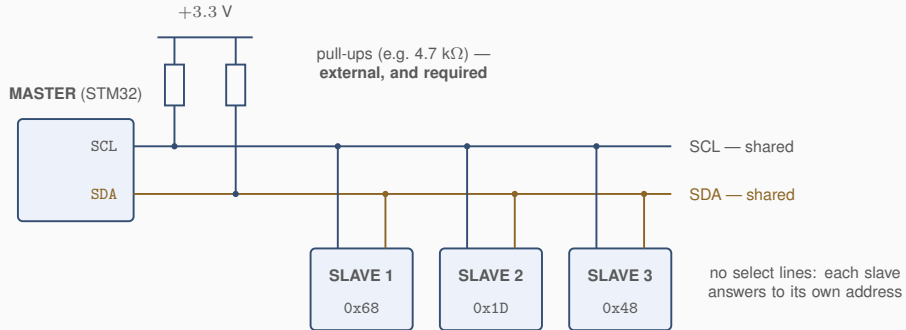


Phase	What happens
START	SDA falls while SCL is high – the one pattern that is otherwise illegal
Address	7 bits plus a read/write bit. Every device compares it against its own
ACK	The addressed device pulls SDA low. Silence means nobody answered
STOP	SDA rises while SCL is high

A missing ACK is the most common I²C fault – and it usually means a **wrong address** or **missing pull-ups**.



SDA may only change while SCL is LOW, so the two transitions that occur while SCL is HIGH are **unambiguous markers** for the start and end of a transfer.



Compare this with SPI's three private chip-select lines: here the pin count stays at **two**, no matter how many devices hang off the bus — selection has moved into **software**.

	SPI	I ² C
Wires	4, plus one CS per device	2 , regardless of count
Speed	Tens of MHz	100 kHz to 1 MHz
Duplex	Full	Half
Addressing	A chip-select line	A 7-bit address
Acknowledge	None — you cannot tell if it worked	ACK per byte
Best for	Displays, flash, fast ADCs	Many slow sensors, EEPROM

The trade in one sentence

SPI buys speed with pins; I²C buys pins with speed — and throws in an acknowledgement, so you know whether the device is actually there.

- **SPI** is four wires and full duplex: a transfer is an **exchange**, so reading requires writing a dummy byte.
- **CPOL** and **CPHA** give four modes — the device datasheet says which.
- $f_{\text{SCK}} = f_{\text{PCLK}}/2^{(\text{BR}+1)}$.
- We use **SPI2**, because SPI1's SCK is PA5 — the user LED.
- **I²C** is two wires and addresses, requiring **open-drain** outputs and external pull-ups.
- A transaction is **START, address+RW, ACK, data, STOP**. A **missing ACK** means wrong address or no pull-ups.
- SPI buys **speed with pins**; I²C buys **pins with speed**.

Next: Chapter 14 — Designing, Building and Verifying

Every peripheral so far, in one program, at the same time.

1 Why This Matters

2 The SPI Bus

3 I²C

4 Design Challenge

Design Challenge To be worked on paper, before any code

The brief

A data logger must connect six devices to one STM32F446RE: an accelerometer sampled at 200 Hz (available in both SPI and I^2C versions), a display controller refreshed 10 times per second with 1024 bytes (SPI only, mode 3, tolerates 8 MHz), a real-time clock read once per second (I^2C only, address $0x68$), a configuration EEPROM written rarely (I^2C only, address $0x50$), and two identical temperature sensors read once per second (I^2C only, address $0x48$ with one address-select pin).

Assign every device to a bus and justify each choice. Then: give the complete pin allocation, checked against the pins already committed to the LED, button, analog input, and serial port; explain how the two identical temperature sensors can coexist; compute the bus utilisation of each bus and confirm both fit; and specify the pull-up resistors, given a bus capacitance of 120 pF. Finally, state which single device you would move to the other bus first if the design turned out to be over budget, and why.

Questions?

Introduction to ARM Cortex-M Microprocessors — Chapter 13